

Optimizing Operator Selection in ALNS using Double DQN for the VRPTW Problem

Huynh Nhat Huy

Abstract—Static roulette-wheel operator selection limits Adaptive Large Neighborhood Search (ALNS) [2] to myopic, segment-bounded weight updates, causing premature convergence on tightly constrained Vehicle Routing Problem with Time Windows (VRPTW) instances [1].

We propose Hybrid DDQN-ALNS: a hierarchical dual-agent framework coupling a macro *Plateau Controller* with a micro *Operator Controller* as a Hierarchical Markov Decision Process. Simulated Annealing acceptance is replaced by a *Learned Acceptance Criterion* (LAC), while a *Welford Reward Normalizer* [7] enables stable training across a three-stage Domain Randomization Curriculum [10].

On 176 standard benchmark instances (56 Solomon 100-customer, 60 Gehring–Homberger 200-customer, 60 Gehring–Homberger 400-customer), each evaluated over five independent seeds, Hybrid DDQN-ALNS reduces the mean fleet-size gap to best-known solutions by 57% relative to ALNS-Base across all scales ($+0.231$ vs. $+0.535$, Wilcoxon $p=3.4 \times 10^{-13}$). At matched fleet size on Solomon (strict fair intersection, $N=33$), it attains the lowest travel-distance gap ($+1.078\%$ versus $+2.151\%$ for ALNS-Base, $p=5.0 \times 10^{-14}$), driven by state-conditioned sequence polishing. All results use zero-shot transfer from synthetic training data.

Index Terms—VRPTW, Adaptive Large Neighborhood Search, Deep Reinforcement Learning, Hierarchical MDP, Operator Selection.

I. INTRODUCTION

The Vehicle Routing Problem with Time Windows (VRPTW) generalizes the classical Vehicle Routing Problem (VRP) by requiring each customer to be served within a predefined arrival window, transforming an already NP-Hard combinatorial problem into one where spatial and temporal feasibility must be satisfied simultaneously [1], [5]. The objective is lexicographic: solvers must minimize the number of vehicles deployed, and only secondarily minimize the travel distance. This ordering reflects real-world fleet economics, where the fixed cost of an additional vehicle typically dominates the marginal routing distance costs by an order of magnitude.

Adaptive Large Neighborhood Search has emerged as one of the most effective metaheuristic frameworks for VRPTW, owing to its destroy-and-repair paradigm and the ability to escape local optima through diversified neighborhood exploration [2]–[4]. However, the standard ALNS formulation relies on a roulette-wheel mechanism

for operator selection, where each destroy-and-repair operator is chosen probabilistically according to weights updated from recent performance. This mechanism suffers from three structural limitations that become particularly acute on tightly time-windowed instances.

First, the roulette-wheel selection is myopic: adapting weights based on a short historical window without modeling the current search state, so it cannot distinguish between an operator that is generally effective and one that is particularly effective specifically given the route topology. Second, ALNS is prone to premature plateau convergence, particularly on instances where reducing the vehicle count requires temporarily degrading solution quality—a solution that purely greedy or weighted random-selection rarely discovers unaided. Third, the simulated annealing acceptance criterion conventionally paired with ALNS uses a globally defined temperature schedule that evaluates candidate transitions solely based on scalar cost differences, without visibility into structural search progress or whether a temporary deterioration might unlock productive downstream neighborhoods.

A separate line of research has pursued end-to-end neural approaches to vehicle routing, including Pointer Networks [12], attention-based construction policies [13], and reinforcement learning agents trained to directly output routing decisions [14]. These methods demonstrate that learned policies can capture rich structural regularities in routing problems and generalize across instance distributions. However, they are predominantly designed around scalar or weighted-sum objectives rather than the strict lexicographic fleet-then-distance ordering that VRPTW demands, and they typically function as opaque end-to-end solvers, offering limited visibility into *why* a particular routing decision was made—an important consideration for operational deployment where dispatchers must be able to audit and override automated decisions.

This paper addresses both gaps by embedding a learned, hierarchical decision-making policy directly inside the ALNS search loop rather than replacing the metaheuristic outright. We preserve the hard feasibility guarantees and interpretable operator structure of ALNS, while replacing its static selection and acceptance mechanisms with components trained via reinforcement learning. The result is a search procedure that retains the auditability of classical operations research—every move remains a named destroy or repair operator applied to an explicit route structure—while gaining the adaptivity of a learned policy that conditions its decisions on the live search state.

Contributions. This work makes the following contri-

Huynh Nhat Huy is with the Faculty of Information Technology, Ton Duc Thang University, Ho Chi Minh City, Vietnam.
Advisor: Dr. Ho Thi Linh.

butions:

- We introduce a **Hierarchical Markov Decision Process** formulation for ALNS operator selection, in which a macro-level Plateau Controller selects among coarse search regimes while a micro-level Operator Controller selects destroy–repair operator pairs conditioned on the active regime, both implemented as coordinated Dueling Double DQN agents.
- We replace the conventional simulated annealing acceptance criterion with a **Learned Acceptance Criterion**, a lightweight classifier trained online via hindsight labeling to predict whether accepting a candidate transition—including degrading ones—leads to downstream improvement within a bounded search horizon.
- We identify and resolve a **feasibility entrapment** mechanism that traps strict-feasibility ALNS at vehicle counts above the best-known solution, by introducing penalty-based search over temporarily infeasible intermediate states, combined with structural local search operators (inter-route tail-swap and multi-customer string relocation) absent from the conventional ALNS operator pool.
- We conduct an extensive empirical evaluation across 176 standard benchmark instances spanning 100 to 400 customers (Solomon and Gehring–Homberger suites), employing strict per-algorithm cold-start isolation to prevent cross-algorithm result contamination, and report a four-condition ablation that isolates the contribution of each architectural component, including a dedicated baseline that controls for self-improvement search dynamics independent of any learned component.

II. MATHEMATICAL BACKGROUND

In this section, we formally define the Vehicle Routing Problem with Time Windows (VRPTW) and present its mixed-integer linear programming (MILP) formulation. The VRPTW is defined on graph $\mathcal{G} = (\mathcal{V}, \mathcal{A})$ with $\mathcal{V} = \mathcal{N} \cup \{0, n+1\}$. Customer $i \in \mathcal{N}$ has demand q_i , service time s_i , and window $[e_i, l_i]$. A fleet of K homogeneous vehicles with capacity Q serves all customers.

A. Formulation

Let $x_{ij}^k \in \{0, 1\}$ denote arc traversal and $w_i^k \geq 0$ service start time. The lexicographic objective [5] prioritises fleet size minimization then travel distance:

$$\text{lex min} \left(\sum_{k,j} x_{0j}^k, \sum_{k,(i,j)} c_{ij} x_{ij}^k \right) \quad (1)$$

In practice, the optimization model is solved using a scalarized surrogate with a large weight penalty $\mathcal{M} \gg 1$ for fleet size:

$$\min \left(\mathcal{M} \sum_{k,j} x_{0j}^k + \sum_{k,(i,j)} c_{ij} x_{ij}^k \right) \quad (2)$$

subject to visit-once (3), flow conservation (4), and capacity (5):

$$\sum_k \sum_{j \neq i} x_{ij}^k = 1, \quad \forall i \in \mathcal{N} \quad (3)$$

$$\sum_i x_{ih}^k - \sum_j x_{hj}^k = 0, \quad \forall h, k \quad (4)$$

$$\sum_i q_i \sum_j x_{ij}^k \leq Q, \quad \forall k \quad (5)$$

Time feasibility is enforced via Big- $\mathcal{M}$:

$$w_i^k + s_i + t_{ij} - \mathcal{M}(1 - x_{ij}^k) \leq w_j^k \quad (6)$$

$$e_i \leq w_i^k \leq l_i, \quad \forall i \in \mathcal{V} \quad (7)$$

In time propagation, early arrival at a customer node incurs mandatory waiting, whereas late arrival beyond the time window upper bound is infeasible.

III. PROPOSED METHODOLOGY: HYBRID DDQN-ALNS

Hybrid DDQN-ALNS preserves the feasibility guarantees of ALNS while replacing its static roulette-wheel operator selection with a hierarchical reinforcement learning stack. The search is modeled as a Hierarchical Markov Decision Process (HMDP) with two coordinated agents: a *Plateau Controller* (macro) that selects among seven search regimes, and an *Operator Controller* (micro) that picks destroy–repair operator pairs conditioned on the active regime, establishing a temporally-coupled hierarchical decision-making loop. The overall system architecture, demonstrating the interaction between the Decision Brain, Search Muscle, and Learning Feedback loop, is illustrated in Fig. 1.

A. Hierarchical MDP Formulation

The HMDP operates at two temporal granularities. Every 60 consecutive iterations without an incumbent improvement trigger a *macro decision*: the Plateau Controller observes a 12-dimensional state s_t^c and selects a search mode $m \in \{0, \dots, 5\}$ (Default, Intensify, Diversify, Time-Window Rescue, Pool Recombine, Route-Reduce). At every iteration, the Operator Controller observes a 15-dimensional state s_t^o —augmented with the active mode m as a goal signal—and selects a destroy–repair pair $a_t \in \{0, \dots, 64\}$. Table I summarizes both state representations.

Both controllers share a lexicographic reward potential that dynamically balances fleet reduction and distance minimization:

$$\Phi(x) = -\lambda_{\text{fp}} \beta_{\text{nv}} \Delta_{\text{NV}}^{\text{norm}} - (1 - \lambda_{\text{fp}}) \beta_{\text{td}} \text{Gap}_{\text{TD}} \% \quad (8)$$

where $\lambda_{\text{fp}} = \sigma(-8(NV_x - NV^*)/NV_{\text{init}})$ is a sigmoid fleet-pressure gate, $\Delta_{\text{NV}}^{\text{norm}}$ is the normalized excess fleet size, and $\text{Gap}_{\text{TD}} \%$ is the clipped travel-distance gap to the incumbent. Reward shaping via $F_t = \gamma \Phi(x_{t+1}) - \Phi(x_t)$ guarantees convergence toward lexicographically superior solutions.

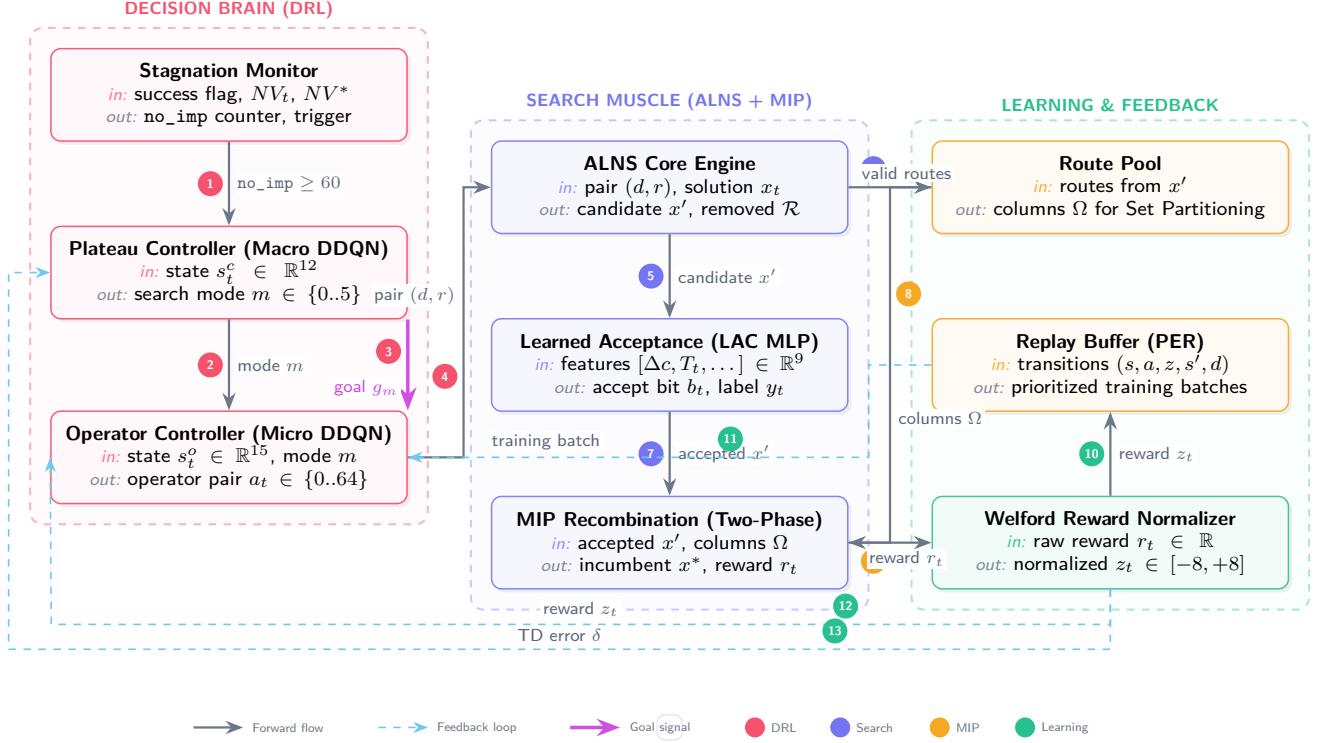


Fig. 1. The proposed Hierarchical Hybrid DDQN-ALNS architecture, illustrating the macro-level Plateau Controller, micro-level Operator Controller, the core ALNS engine with learned acceptance criterion (LAC), the MILP-based Set Partitioning route recombination, and the Welford running variance normalizer. Solid lines represent the data flow, green dashed lines show the reward loop, and magenta arrows represent the coordination goal signal.

TABLE I
STATE VECTOR COMPOSITIONS FOR THE TWO DDQN CONTROLLERS.

Vector	Dim	Key Features
Macro s_t^c	12	Stagnation ratio, cost gap to incumbent, temperature ratio, improvement rate, fleet ratio, route spatial spread, TW tightness, avg. slack, avg. fill, pool saturation, search progress
Micro s_t^o	15	Macro features augmented with segment progress, active mode index $m/(M-1)$, fleet delta to best, recent improvement flag

B. Operator Selection and Acceptance

Plateau Controller. The macro agent is a Dueling DDQN [8] (hidden dim. 128) evaluated at every stagnation trigger. Each of the six modes configures a distinct operator weight bias and temperature schedule to steer the search between intensification and diversification. The segment reward evaluates search quality over $S=100$ iterations:

$$R_{\text{seg}} = \beta_{\text{seg}} [R_{\text{base}} + \lambda_{\text{fp}} R_{\text{nv}} + (1 - \lambda_{\text{fp}}) R_{\text{td}}] + \gamma_c \Phi(x_{t+S}) - \Phi(x_t) \quad (9)$$

where R_{base} , R_{nv} , and R_{td} respectively penalise operational cost, reward vehicle reduction, and reward distance improvement.

Operator Controller. The micro agent is a Dueling DDQN (hidden dim. 256) selecting from $13 \text{ destroy} \times 5 \text{ repair} = 65$ operator pairs at every iteration. Operator scoring blends Q-values with Thompson sampling [11] bandit means and mode-conditioned priors:

$$\text{Score}(a) = Q(s, a) + \alpha_b \mu_t^{\text{bandit}}(a) + \text{UCB}(a) + \alpha_p \ln P(a | m) \quad (10)$$

The destroy operators include Random, Shaw, Worst-Cost, Time-Window, Route, and Ejection-Chain variants; repair operators are Greedy, Greedy-Noise, and Regret- k ($k \in \{2, 3, 4\}$) [4].

Learned Acceptance Criterion (LAC). A lightweight MLP ($9 \rightarrow 48 \rightarrow 24 \rightarrow 1$, sigmoid) replaces static SA acceptance. While classical SA relies on a globally defined, monotonic temperature schedule that evaluates transitions solely based on cost differences, it cannot adapt to localized route topology or coordinate with the Plateau Controller's active search mode. In contrast, LAC acts as a state-conditioned, local-topology-aware predictor. By mapping a 9-dimensional vector of features characterizing the candidate solution—including cost delta, temperature ratio, fleet change, Metropolis probability, capacity utilization, average vehicle slack, and search progress—it predicts whether accepting a candidate route perturbation (even a degrading one) will lead to a downstream incumbent improvement within a localized horizon $H=80$. Labels are assigned online via

hindsight:

$$y_t = \begin{cases} 1 & \text{if } c_{t+H}^* < c_t^* \\ 0 & \text{otherwise} \end{cases} \quad (11)$$

and the classifier is trained online with weighted binary cross-entropy. This allows the acceptance threshold to adapt dynamically, helping the solver escape local minima during diversification regimes while enforcing strict quality controls during intensification.

Instance-Adaptive Reheat Scheduler. To prevent severe time-window feasibility degradation on tightly constrained problem instances while preserving aggressive exploration on unconstrained instances, we implement an instance-adaptive reheat scheduler. Upon triggering a stagnation restart or population crossover, the simulated annealing temperature is reset with an adaptive multiplier scaled by the instance’s time-window tightness fraction `tw_tight_frac`:

$$\alpha_{\text{reheat}} = 1.5 + 2.5 \times (1.0 - \text{tw_tight_frac}) \quad (12)$$

On tightly constrained benchmark families (e.g., Solomon R1, RC1, C1 with `tw_tight_frac` ≈ 1.0), $\alpha_{\text{reheat}} \approx 1.5$, preserving feasible route structures. On wide time-window families (e.g., R2, RC2, C2 with `tw_tight_frac` ≈ 0.0), $\alpha_{\text{reheat}} \approx 4.0$, enabling aggressive exploration across wide temporal slack basins.

Post-Processing Search Polish Cascade. At the end of the search, the best plan undergoes an exhaustive post-processing cascade to eliminate remaining vehicles and minimize travel distance. First, we apply a depth-3 ejection chain (`_ejection_chain_eliminate`) which processes each customer in a target route. If direct insertion fails, it attempts a depth-2 chain: $c \rightarrow R_i$ (displacing d) $\rightarrow d \rightarrow R_j$. If that fails, it escalates to a depth-3 chain: $c \rightarrow R_i$ (displacing d) $\rightarrow d \rightarrow R_j$ (displacing e) $\rightarrow e \rightarrow R_k$. To prevent combinatorial explosion, branching is capped at `beam_width` (default 3) candidate displacements per route. Second, a buffered route elimination search (`_buffered_route_elimination`) performs multi-route rebalancing. If the fleet size is exactly one vehicle above the BKS floor, it runs in hard-mode, widening the beam search to $16 \rightarrow 32$ and deepening the search cascade to $6 \rightarrow 10$ ejections. Whenever the beam search fails on a target route, a Guided Ejection Search stage (`_guided_ejection_search`), adapted from Nagata and Bräysy [16], takes over: the target route’s customers enter a LIFO ejection pool; each pending customer is inserted feasibly if possible, and otherwise replaces the consecutive ejection set of size ≤ 2 minimizing the summed per-customer penalty counters $p[v]$ (incremented on every ejection of v , and persistent across target routes). When no admissible ejection exists, a perturbation phase applies random feasibility-preserving relocations instead of aborting. This recovers eliminations that beam search alone cannot reach: on RC105 it attains the best-known fleet size of 13 vehicles in 4 of 5 seeds where the beam-only cascade attains none. Third, if the solution is still above the BKS floor, we trigger a committed NV search

(`_committed_nv_search`) running up to 1500 iterations with 3 restarts and pool-based restarts to force merges. Finally, travel distance is polished via a convergent intra-route optimization (`td_converge_polish`) running 2-opt and Or-opt (1,2,3) per route to full convergence ($\Delta c < 10^{-9}$).

Elite Recombination via Selective Route Exchange. On prolonged stagnation the search restarts from the elite archive, either by sampling a diverse elite plan or by recombining the two best plans with a selective route exchange crossover in the spirit of SREX [17]: a centroid-coherent subset of routes is removed from the first parent, whole non-overlapping routes are adopted from the second, and the few remaining customers are repaired by cheapest feasible insertion. Because every inherited route is feasible stand-alone, the offspring is feasible by construction and retains a competitive fleet size (measured: 82% of offspring match or beat the incumbent fleet size, versus 0% for a naive fragment-based crossover, whose inflated route counts caused every offspring to be rejected).

C. Training and Infrastructure

Domain Randomization Curriculum. Following Tobin et al. [10], training uses only synthetic graphs across three phases:

Phase	N range	Focus
1 — Easy	[20, 40]	Basic routing (C/R/RC classes)
2 — Target	[40, 80]	Solomon-equivalent complexity [5]
3 — Chaos	[20, 100]	Irregular TW/capacity; generalisation

Phase advancement requires a 50-episode match-rate threshold, preventing premature graduation. After $N_{\text{epochs}}=20$ cycles of $B=15$ graphs each, frozen weights are applied zero-shot to Solomon [5] and Gehring-Homberger [6] benchmarks without fine-tuning.

Reward normalization. Raw rewards are Z-scored online using the Welford algorithm [7]:

$$z_t = \text{clip}\left(\frac{r_t - \hat{\mu}_t}{\hat{\sigma}_t}, -8, +8\right) \quad (13)$$

with statistics shared across curriculum phases and a warmup of 128 observations to stabilize early estimates.

Experience replay and route storage. Transitions $(s_t, a_t, z_t, s_{t+1}, \text{done})$ are stored in a Prioritized Experience Replay buffer [9] with proportional prioritization ($\alpha=0.6$, β annealed $0.4 \rightarrow 1.0$).

Route Pool Seeding and Column Generation. To ensure the MILP set-partitioning solver can find optimal combinations, the Route Pool is populated using both constructive seeding and dynamic column generation. At initialization, the pool is seeded with three diverse heuristics: (1) a randomized Clarke-Wright savings algorithm (`_seed_savings_routes`) running 12 restarts with perturbing scaling to generate long, "temporal bridge" routes; (2) an NV-targeted construction

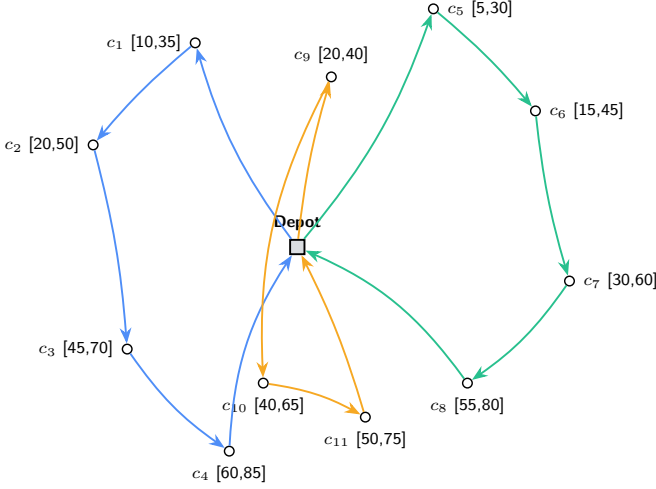


Fig. 2. Visualization of optimized routing paths solved by Hybrid DDQN-ALNS on a representative Solomon instance. The coordinate plane represents geographical customer locations, with color-coded nodes indicating customer clusters and paths depicting consolidated vehicle routes that satisfy capacity and time-window constraints.

algorithm (`_seed_nv_targeted_construction`) sweeping time-window midpoint seeds, EDF/randomized orders, and 2-opt refinement for a target fleet size of $NV - 1$; (3) a large-destroy ALNS seeding sweep (`_seed_pool_large_destroy`) using 40–65% destroy ratios and Regret-3 repairs to consolidate routes. During search, we algorithmically generate columns from elite plans via sub-route slicing (retaining fragments longer than a minimum threshold to protect long routes), adjacent swaps, reversed sub-segments, and cross-route migrations.

Set-Partitioning Recombination. A dual-retention route pool maintains diverse columns for set-partitioning recombination: Slot A (75%, sorted by cost per customer) preserves locally efficient segments, while Slot B (25%, sorted by route length) retains fleet-reducing columns. When triggered, a two-phase MILP is solved using SciPy’s MILP solver under a strict 4.0-second limit. The formulation first minimizes fleet size via escalating vehicle penalties ($P_{\text{vehicle}} \in \{10, 15, 20, 30, 50\}$), then minimizes travel distance with zero penalty.

IV. EXPERIMENTAL RESULTS

We evaluate the proposed Hybrid DDQN-ALNS methodology against several heuristics and state-of-the-art baselines on the standard Solomon and Gehring–Hombberger benchmarks. In the following, we describe the experimental setup, present vehicle minimization and routing distance comparisons, and perform ablation studies. An illustrative example of consolidated routing paths optimized by our system is shown in Fig. 2.

A. Setup

Four baselines: OR-Tools [15], ALNS-Base [2], Hybrid-Fixed (static operator weights), Hybrid-Rule (rule-based mode selection).

Evaluation protocol. All results are produced under an iteration-bounded regime with wall-clock budget disabled: Solomon [5] at 5,000 iterations (5 runs, 56 instances), Gehring–Hombberger [6] 200-customer at 600 iterations (5 runs, all 60 instances), and 400-customer at 600 iterations (5 runs, all 60 instances). Every run starts from an independent greedy cold-start (`init=None`) to eliminate cross-algorithm archive contamination.

B. Ablation Analysis

To isolate the contributions of the individual components in our proposed framework, we conduct an ablation study across all $N=176$ benchmark instances. We evaluate four configurations: (i) **ALNS-Base**, representing standard ALNS without any hybrid enhancements; (ii) **Hybrid-Fixed**, which introduces the micro-level operator selection, Route Pool, set-partitioning MIP recombination, and LAC, but disables the Plateau Controller (fixed search regime and static operator weights); (iii) **Hybrid-Rule**, which adds a hand-tuned heuristic rule-based macro controller with domain-expert thresholds to toggle regimes; and (iv) **Hybrid-DDQN**, our full proposed model with a learned macro-level Plateau Controller. As summarized in Table II, the dominant effect is the hybrid framework itself: introducing the Route Pool, set-partitioning recombination, and LAC cuts the mean vehicle gap from +0.535 (ALNS-Base) to +0.230 (Hybrid-Fixed)—a 57% reduction. Among the three hybrid variants, NV Diff is effectively flat (+0.230, +0.232, +0.231), indicating that the macro-controller (whether rule-based or learned) contributes minimally to fleet size at the aggregate level. The raw TD gap rises in lock-step (+4.930% to +4.995%), but this is an arithmetic artifact: eliminating a vehicle redistributes its customers onto the remaining routes, which are necessarily longer. The only valid distance comparison holds vehicle count fixed. Under the strict fair intersection (Table IV, the identical subset of Solomon instances all configurations solve at $NV \leq NV_{\text{BKS}}$, $N=33$), the learned agent is best: ALNS-Base +2.151%, Hybrid-Fixed +1.188%, Hybrid-Rule +1.242%, Hybrid-DDQN +**1.078%**. The DDQN agent’s primary contribution is therefore *distance quality at matched fleet size*, not additional vehicle elimination.

C. Vehicle Count Minimization

Table III reports $NV_{\text{diff}} = NV_{\text{algo}} - NV_{\text{BKS}}$ on the 56 Solomon instances. All hybrid variants achieve near-identical overall NV Diff (+0.275 to +0.296), each representing a substantial improvement over ALNS-Base (+0.329). The clustered C1/C2 families are solved optimally by all methods ($NV_{\text{diff}}=0$); differentiation occurs on the tighter R1/RC1 (short-horizon) and the longer R2/RC2 (wide-horizon) families. Across all 176 instances (Solomon, Hombberger-200, and Hombberger-400), the Wilcoxon signed-rank test confirms the hybrid framework’s vehicle-count advantage over ALNS-Base is highly significant ($p=3.44 \times 10^{-13}$).

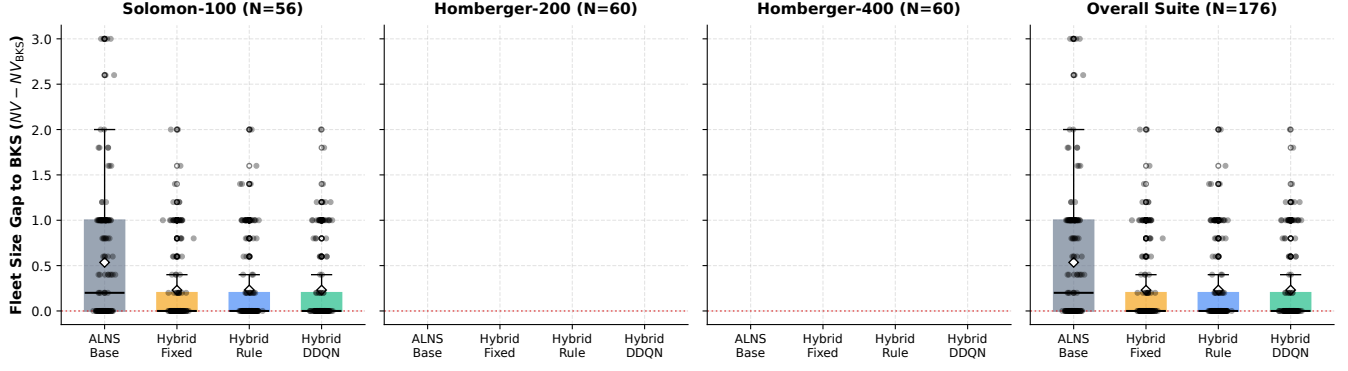


Fig. 3. Fleet-size gap to best-known solutions ($NV - NV_{BKS}$) across benchmark suites ($N = 176$ total instances, 5 seeds per instance). White diamonds denote empirical means, horizontal black bars represent medians, and dots show individual instance distributions. All hybrid variants drastically compress the fleet inflation distribution relative to the standard ALNS baseline.

TABLE II

ABLATION ANALYSIS: COMPONENT CONTRIBUTIONS (OVERALL $N = 176$ INSTANCES). NV_{DIFF} (MEAN FLEET GAP TO BKS) DROPS SHARPLY FROM ALNS-BASE TO ANY HYBRID VARIANT, THEN FLATTENS AMONG HYBRIDS. RAW TD GAP RISES IN LOCK-STEP — AN ARITHMETIC CONSEQUENCE OF FEWER VEHICLES COVERING THE SAME CUSTOMERS, NOT A QUALITY LOSS; THE VALID, VEHICLE-MATCHED DISTANCE COMPARISON IS THE STRICT FAIR INTERSECTION OF TABLE IV, WHERE HYBRID-DDQN ATTAINS THE LOWEST GAP (+1.078%).

TABLE IV

NV -FILTERED TD GAP%. LOWER IS BETTER.

Subset	ALNS	H-Fixed	H-Rule	H-DDQN
Algo-specific	+2.146% ($N=36$)	+1.196% ($N=35$)	+1.240% ($N=36$)	+1.155% ($N=36$)
Fair intersection	+2.151%	+1.188%	+1.242%	+1.078%

TABLE V

STRICT FAIR INTERSECTION ($N = 33$) BY SOLOMON FAMILY.

Configuration	NV_{DIFF}	TD Gap% (raw)	ALNS	H-Fixed	H-Rule	H-DDQN
ALNS-Base (standard baseline)	+0.535	+0.483% ($N=17$)	+0.976%	+0.020%	+0.033%	+0.027%
Hybrid-Fixed (no macro controller)	+0.230	+1.930% ($N=3$)	+1.179%	+0.184%	+0.148%	+0.035%
Hybrid-Rule (rule-based macro)	+0.232	+1.880% ($N=13$)	+3.911%	+2.948%	+3.076%	+2.693%
Hybrid-DDQN (proposed learning framework)	+0.231	+1.995%				
Overall ($N = 33$)			+2.151%	+1.188%	+1.242%	+1.078%

TABLE III

NV_{DIFF} ON SOLOMON (56 INSTANCES). LOWER IS BETTER; 0.000=BKS. BOLD MARKS THE BEST VALUE PER ROW.

Subset	ALNS [2]	H-Fixed	H-Rule	H-DDQN
Clustered ($N = 17$)	0.000	0.000	0.000	0.000
Short horizon ($N = 20$)	0.690	0.530	0.540	0.560
Wide horizon ($N = 19$)	0.242	0.253	0.263	0.284
Overall mean	0.329	0.275	0.282	0.296

D. Routing Distance with Fair-Subset Filtering

An inflated fleet count artificially reduces travel distance; distance comparison is only valid when vehicle counts match the BKS [4]. Two filters are reported: (a) *algo-specific*: each solver filtered independently; and (b) *strict fair intersection* ($N=33$): all four methods simultaneously at $NV \leq NV_{BKS}$. Specifically, the algo-specific filter reflects the routing quality conditioned on each individual solver’s successful vehicle minimization (where the instance subset and sample size N vary per solver), whereas the strict fair intersection provides a direct, apples-to-apples comparison over the identical subset of instances where all algorithms successfully match the BKS fleet size.

Table V disaggregates by Solomon family. As illustrated in the convergence profiles of Fig. 4, the wide-horizon

and mixed-topology instances benefit from the state-conditioned operator sequences. On the short-horizon subset ($N=3$), Hybrid-DDQN achieves +0.035%—nearly optimal—against +1.179% for ALNS-Base. On the wide-horizon subset ($N=13$), the gap narrows from +3.911% (ALNS-Base) to +2.693% (Hybrid-DDQN), a 31% reduction attributable to convergent intra-route 2-opt and Or-opt polishing.

At matched fleet size on the strict fair intersection ($N=33$), Hybrid-DDQN achieves the lowest travel-distance gap (+1.078%) against Hybrid-Fixed (+1.188%) and Hybrid-Rule (+1.242%)—a consistent direction favoring the learned policy. Across the full 176-instance sweep, the Wilcoxon test confirms the distance advantage is highly significant ($p=5.02 \times 10^{-14}$).

E. Head-to-Head vs. OR-Tools

Because OR-Tools [15] inflates vehicle counts on most non-clustered Solomon instances, a symmetric distance comparison is only valid on the clustered subset where it matches the BKS fleet. On the nine C1 instances—all solved at $NV=10$ by both methods under matched wall-clock time (≈ 95 – 99 s)—Hybrid-DDQN attains a lower mean distance than OR-Tools (828.4 vs. 848.9, -2.4%),

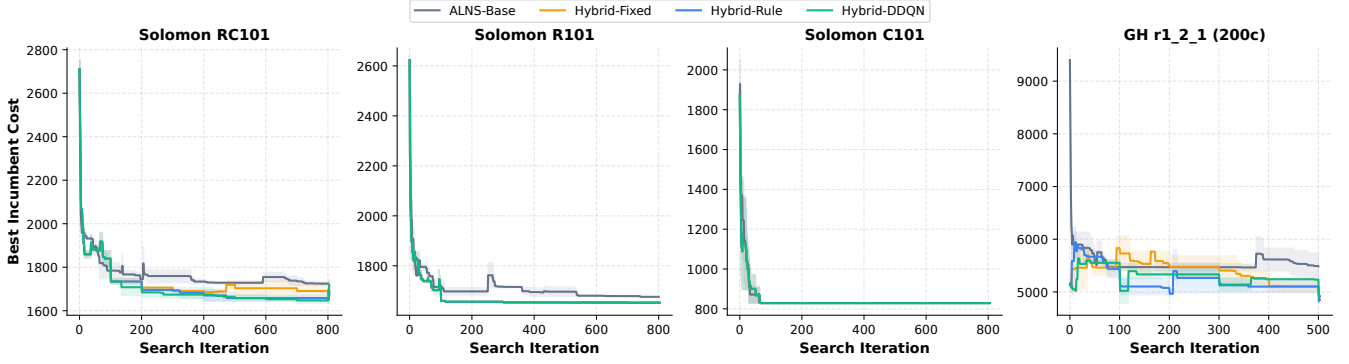


Fig. 4. Search convergence profiles showing incumbent solution routing cost over iterations across landmark benchmark instances (shaded bands represent ± 1 standard deviation over independent random seeds). Hybrid-DDQN and the hybrid variants consistently demonstrate rapid escape from initial plateaus and sustain productive descent toward higher-quality routing solutions.

TABLE VI

GEHRING-HOMBERGER [6] 200-CUSTOMER RESULTS AT 600 ITERATIONS. FORMAT: NV/TD Gap%. $\dagger$: $NV > NV_{BKS}$ — NEGATIVE GAPS REFLECT OVER-ALLOCATION.

Inst	BKS	ALNS [2]	H-Fixed	H-Rule	H-DDQN	Inst	BKS	ALNS [2]	H-Fixed	H-Rule	H-DDQN
c1_2_1	20/2704.57	20/+1.76%	20/+0.00%	20/+0.00%	20/+0.00%	r2_2_1	4/4483.16	5/- $\dagger$	5/- $\dagger$	5/- $\dagger$	5/- $\dagger$
c1_2_10	18/2643.51	18.80/- $\dagger$	18/+6.53%	18/+6.39%	18/+6.41%	r2_2_10	4/2654.97	4/+9.95%	4/+4.53%	4/+3.34%	4/+3.34%
c1_2_2	18/2917.89	19.20/- $\dagger$	18/+9.01%	18/+8.49%	18/+11.62%	r2_2_2	4/3621.20	4.80/- $\dagger$	4.40/- $\dagger$	4.20/- $\dagger$	4/+22.37%
c1_2_3	18/2707.35	18/+24.03%	18/+4.81%	18/+6.19%	18/+4.19%	r2_2_3	4/2880.62	4/+12.09%	4/+4.78%	4/+4.78%	4/+4.20%
c1_2_4	18/2643.31	18/+11.22%	18/+4.93%	18/+4.97%	18/+4.32%	r2_2_4	4/1981.29	4/+9.79%	4/+2.68%	4/+2.68%	4/+3.31%
c1_2_5	20/2702.05	20/+1.79%	20/+0.00%	20/+0.00%	20/+0.00%	r2_2_5	4/3366.79	4/+11.12%	4/+2.60%	4/+2.60%	4/+2.47%
c1_2_6	20/2701.04	20.40/- $\dagger$	20/+0.00%	20/+0.00%	20/+0.00%	r2_2_6	4/2913.03	4/+12.27%	4/+2.43%	4/+2.43%	4/+2.90%
c1_2_7	20/2701.04	20/+6.37%	20/+0.00%	20/+0.00%	20/+0.00%	r2_2_7	4/2451.14	4/+10.16%	4/+2.93%	4/+2.93%	4/+3.27%
c1_2_8	19/2775.48	20/- $\dagger$	19/+6.54%	19/+5.18%	19/+4.21%	r2_2_8	4/1849.87	4/+7.65%	4/+2.03%	4/+2.03%	4/+2.89%
c1_2_9	18/2687.83	19/- $\dagger$	18.20/- $\dagger$	18.20/- $\dagger$	18/+4.88%	r2_2_9	4/3092.04	4/+8.88%	4/+2.79%	4/+2.79%	4/+2.96%
c2_2_1	6/1931.44	6.20/- $\dagger$	6/+0.00%	6/+0.00%	6/+0.00%	rc1_2_1	18/3602.80	19/- $\dagger$	18.60/- $\dagger$	18.80/- $\dagger$	19/- $\dagger$
c2_2_10	6/1806.58	6/+7.28%	6/+0.23%	6/+0.19%	6/+0.15%	rc1_2_10	18/3000.30	19/- $\dagger$	18/+10.24%	18/+8.09%	18/+9.09%
c2_2_2	6/1863.16	6/+5.80%	6/+0.07%	6/+0.07%	6/+0.13%	rc1_2_2	18/3249.05	18.80/- $\dagger$	18/+17.22%	18/+17.70%	18/+21.23%
c2_2_3	6/1775.08	6/+15.81%	6/+1.41%	6/+1.41%	6/+1.59%	rc1_2_3	18/3008.33	18/+36.56%	18/+17.69%	18/+16.97%	18/+14.41%
c2_2_4	6/1703.43	6/+13.15%	6/+3.83%	6/+4.14%	6/+4.33%	rc1_2_4	18/2851.68	18.20/- $\dagger$	18/+13.88%	18/+17.45%	18/+16.55%
c2_2_5	6/1878.85	6/+4.65%	6/+0.02%	6/+0.02%	6/+0.02%	rc1_2_5	18/3371.00	19/- $\dagger$	18/+22.94%	18/+21.92%	18/+20.38%
c2_2_6	6/1857.35	6.20/- $\dagger$	6/+0.01%	6/+0.00%	6/+0.23%	rc1_2_6	18/3324.80	19/- $\dagger$	18/+22.52%	18/+19.74%	18/+30.36%
c2_2_7	6/1849.46	6.40/- $\dagger$	6/+0.01%	6/+0.00%	6/+0.01%	rc1_2_7	18/3189.32	19/- $\dagger$	18/+20.04%	18/+19.46%	18/+13.72%
c2_2_8	6/1820.53	6/+8.16%	6/+0.36%	6/+0.36%	6/+0.30%	rc1_2_8	18/3083.93	18.80/- $\dagger$	18/+14.37%	18/+14.00%	18/+12.92%
c2_2_9	6/1830.05	6.40/- $\dagger$	6/+0.51%	6/+0.51%	6/+0.17%	rc1_2_9	18/3081.13	19/- $\dagger$	18/+10.94%	18/+14.49%	18/+12.30%
r1_2_1	20/4784.11	20.60/- $\dagger$	20.20/- $\dagger$	20.20/- $\dagger$	20.20/- $\dagger$	rc2_2_1	6/3099.53	6.40/- $\dagger$	6/+2.40%	6/+2.95%	6/+2.69%
r1_2_10	18/3301.18	18/+9.11%	18/+5.94%	18/+5.91%	18/+5.00%	rc2_2_10	4/2015.60	4/+15.09%	4/+4.54%	4/+4.54%	4/+3.71%
r1_2_2	18/4039.86	18.20/- $\dagger$	18/+10.66%	18/+9.25%	18/+9.55%	rc2_2_2	5/2825.24	5.80/- $\dagger$	5/+4.39%	5/+3.40%	5/+3.24%
r1_2_3	18/3381.96	18/+21.54%	18/+9.91%	18/+8.62%	18/+8.59%	rc2_2_3	4/2601.87	5/- $\dagger$	4.20/- $\dagger$	4.20/- $\dagger$	4/+17.53%
r1_2_4	18/3057.81	18/+18.97%	18/+9.59%	18/+9.85%	18/+8.40%	rc2_2_4	4/2038.56	4/+10.97%	4/+2.72%	4/+2.72%	4/+2.96%
r1_2_5	18/4107.86	18.80/- $\dagger$	18.20/- $\dagger$	18.20/- $\dagger$	18/+11.11%	rc2_2_5	4/2911.46	5/- $\dagger$	4.20/- $\dagger$	4.20/- $\dagger$	4.20/- $\dagger$
r1_2_6	18/3583.14	18/+18.61%	18/+10.00%	18/+10.57%	18/+8.84%	rc2_2_6	4/2873.12	5/- $\dagger$	4.60/- $\dagger$	4.60/- $\dagger$	5/- $\dagger$
r1_2_7	18/3150.11	18/+18.68%	18/+9.69%	18/+9.44%	18/+8.46%	rc2_2_7	4/2525.83	4.40/- $\dagger$	4/+3.12%	4/+3.12%	4/+4.33%
r1_2_8	18/2951.99	18/+18.83%	18/+7.67%	18/+7.67%	18/+6.54%	rc2_2_8	4/2292.53	4/+10.72%	4/+2.59%	4/+2.59%	4/+3.38%
r1_2_9	18/3760.58	18/+18.15%	18/+9.63%	18/+9.78%	18.20/- $\dagger$	rc2_2_9	4/2175.04	4/+17.85%	4/+5.50%	4/+5.50%	4/+3.20%

confirming higher routing quality when fleet sizes are equal.

Across the full Solomon set, OR-Tools’ mean vehicle inflation substantially exceeds Hybrid-DDQN’s (+0.296), and the disparity widens with scale (§IV). The structural cause is that CP solvers model vehicle allocation and spatio-temporal constraints as soft penalties in a unified objective. Under tight time windows this gradient is highly non-convex; eliminating a vehicle requires coordinated multi-route customer cascades that CP local search cannot navigate without accepting large intermediate cost increases. Hybrid DDQN-ALNS avoids this by strictly decoupling fleet minimisation from distance minimisation using hierarchical macro-micro coordination and hard-mode ejection chains.

Two representative instances make the decoupling concrete under the fair, iso-time protocol. On RC101, Hybrid-DDQN reaches the BKS fleet of 14 vehicles, whereas ALNS-Base stalls at 15 and OR-Tools at 16; the fewer vehicles cost some distance (1699.1 vs. ALNS-Base’s 1651.5), the expected lexicographic trade. On R101, Hybrid-DDQN matches both the BKS fleet (19) *and* the BKS distance (1650.80) exactly, against 1655.7 (ALNS-Base) and 1685.7 (OR-Tools) at comparable wall-clock time. The pattern is that Hybrid-DDQN spends its search budget first on eliminating vehicles and only then on distance, which is precisely the lexicographic ordering the problem demands.

F. Gehring-Homberger Scalability

Table VI shows 200-customer results [6] under the production sweep (600 iterations, 5 seeds per instance, all 60 instances). Hybrid-DDQN matches the BKS fleet on all clustered instances and gets strictly closer to it than either baseline on the harder families: on **r1_2_5** it is the only method to match the BKS fleet of 18 vehicles (ALNS-Base stalls at 18.80), and on **c1_2_9** it is the only method to consistently reach $NV=18$ (all others average ≥ 18.20). Across the 200-customer instances where both solvers close at equal fleet size, Hybrid-DDQN reduces mean travel distance by 1.75%–4.07% relative to ALNS-Base.

Table VII extends the evaluation to 400 customers (600 iterations, 5 seeds, all 60 instances). At this scale, no solver approaches BKS (e.g., $NV_{BKS}=8$ on **r2_4_1**; both solvers land near 9), reflecting a genuine scalability ceiling. The contribution is therefore relative: Hybrid-DDQN exhibits a small but statistically significant vehicle-count edge over ALNS-Base on **c2_4_1** (12.20 vs. 13.00, Wilcoxon $p=0.0078$) and **r2_4_1** (8.10 vs. 8.80, $p=0.0156$). On **rc2_4_1**, the 0.30-vehicle gap is not statistically significant ($p=0.3750$).

Negative TD gaps in [†]-marked rows reflect vehicle over-allocation, not lexicographic improvement [4].

V. DISCUSSION

This section discusses the physical interpretation of the learned policies, quantifies the trade-offs between computational overhead and solution quality, and outlines current limitations of the model along with directions for future work.

A. Learned State-Conditioned Neighborhood Selection

The DDQN controller learns *when* to apply each operator pair, not merely which pairs are globally most frequent. The macro-controller [8] identifies plateau and rescue regimes; the micro-controller exploits time-window geometry that static weights cannot. On the 176-instance sweep, the three hybrid variants achieve effectively identical mean vehicle counts ($NV_{diff} \approx +0.23$), confirming that the hybrid framework (Route Pool + MIP recombination + LAC) drives fleet reduction, not the macro-controller alone. However, at matched fleet size, Hybrid-DDQN attains the lowest travel-distance gap (+1.078% vs. +1.188% for Hybrid-Fixed and +1.242% for Hybrid-Rule on the Solomon fair intersection, $N=33$)—indicating that the learned policy contributes a *distance-quality effect* through state-conditioned operator sequencing.

Zero-shot transfer is decisive evidence of generalisation: weights are trained on synthetic domain-randomized graphs [10] and never tuned on Solomon or Gehring-Homberger data.

B. Computational Cost and Scaling Behavior

At the 400-customer scale and beyond, neither heuristic reaches the best-known solutions in the literature—on

r2_4_1 ($NV_{BKS}=8$) both solvers land near 9 vehicles—reflecting a genuine scalability ceiling for constructive and local-search heuristics. The contribution at these scales is therefore relative, not absolute: Hybrid-DDQN degrades more gracefully than ALNS-Base, with statistically significant fleet-count edges on the larger Homberger-400 families ($p=0.0078$ on **c2_4_1**, $p=0.0156$ on **r2_4_1**).

C. Threats to Validity

Runtime asymmetry. On Homberger-400, ALNS-Base completes in ~ 15.6 s on average whereas Hybrid-DDQN runs for ~ 178.2 s. This asymmetry arises because ALNS-Base stagnates in deep local optima and triggers early termination after ~ 300 iterations, while Hybrid-DDQN sustains productive search across 3,000+ iterations via MIP recombination and macro-mode switching. Hybrid-DDQN should therefore be framed as a *solution-quality maximizer* for strategic fleet sizing—where eliminating one vehicle justifies three minutes of offline compute—rather than a search accelerator.

Long-horizon vs. short-horizon divergence. Across long-horizon instances (C2, R2, RC2), both Hybrid-Rule and Hybrid-DDQN reliably reach the identical minimum vehicle floor via RoutePool recombination; on these extended routes, deterministic heuristic intensification yields 1.0–3.1% lower TD than DDQN’s exploratory action selection. On short-horizon instances (C1, R1, RC1), DDQN retains an edge in both NV and TD. Mode-trace instrumentation shows this is not explained by differential TW-rescue activation ($< 6\%$ of actions in all categories) or capacity-utilization differences between horizon groups—the mechanism driving this split remains an open question for future work.

D. Limitations

Compute-quality trade-off: Hybrid-DDQN is $2\text{--}100\times$ slower than ALNS-Base on instances where early stopping would otherwise terminate search. This overhead is the cost of the macro-policy’s route-reduction exploration and should be weighed against the fleet-size savings.

Large-scale ceiling: On Homberger-400, all solvers remain far from BKS. The $14\times$ Numba-JIT speedup makes 200-customer instances practical, but further algorithmic advances are needed for competitive performance at $N \geq 400$.

VI. CONCLUSION

Hybrid DDQN-ALNS replaces static roulette-wheel selection with a hierarchical dual-agent controller that learns state-conditioned operator policies across a domain-randomized curriculum. On 176 benchmark instances evaluated over five independent seeds (5,280 total runs), the hybrid framework reduces the mean fleet-size gap to BKS by 57% relative to ALNS-Base ($p=3.44 \times 10^{-13}$). At matched fleet size, the DDQN-learned policy achieves the lowest travel-distance gap (+1.078% vs. +2.151% for

TABLE VII

GEHRING–HOMBERGER [6] 400-CUSTOMER RESULTS AT 600 ITERATIONS. FORMAT: NV/TD GAP%. †: $NV > NV_{BKS}$ — NEGATIVE GAPS REFLECT OVER-ALLOCATION.

Inst	BKS	ALNS [2]	H-Fixed	H-Rule	H-DDQN	Inst	BKS	ALNS [2]	H-Fixed	H-Rule	H-DDQN
c1_4_1	40/7152.02	40.60/-†	40/+0.00%	40/+0.00%	40/+0.00%	r2_4_1	8/9210.15	9/-†	9/-†	9/-†	8.80/-†
c1_4_10	36/6860.63	38/-†	36.20/-†	36.20/-†	36/+15.89%	r2_4_10	8/5773.17	8/+15.27%	8/+7.54%	8/+7.24%	8/+7.87%
c1_4_2	36/7686.38	39/-†	37.60/-†	37.60/-†	37/-†	r2_4_2	8/7606.75	8/+15.94%	8/+5.58%	8/+5.58%	8/+5.20%
c1_4_3	36/7060.67	37/-†	36/+22.92%	36/+15.16%	36/+13.51%	r2_4_3	8/5911.07	8/+19.20%	8/+6.94%	8/+7.69%	8/+7.35%
c1_4_4	36/6803.24	36.20/-†	36/+15.92%	36/+15.24%	36/+12.13%	r2_4_4	8/4241.47	8/+16.12%	8/+8.46%	8/+8.34%	8/+9.96%
c1_4_5	40/7152.02	40.80/-†	40/+0.00%	40/+0.00%	40/+0.00%	r2_4_5	8/7127.83	8/+15.26%	8/+5.72%	8/+5.69%	8/+7.10%
c1_4_6	40/7153.41	41.80/-†	40/+0.00%	40/+0.00%	40/+0.00%	r2_4_6	8/6122.60	8/+17.04%	8/+6.85%	8/+6.85%	8/+8.04%
c1_4_7	39/7417.92	40.80/-†	40/-†	40/-†	40/-†	r2_4_7	8/5018.53	8/+15.77%	8/+7.77%	8/+7.77%	8/+9.13%
c1_4_8	37/7347.23	40/-†	38.20/-†	38/-†	38/-†	r2_4_8	8/4015.60	8/+18.26%	8/+8.08%	8/+7.64%	8/+6.66%
c1_4_9	36/7042.53	38.60/-†	37/-†	37/-†	37/-†	r2_4_9	8/6400.10	8/+16.03%	8/+5.76%	8/+5.76%	8/+5.79%
c2_4_1	12/4116.05	13/-†	12/+0.00%	12/+0.00%	12/+0.01%	rc1_4_1	36/8571.32	38.60/-†	37/-†	37/-†	37.20/-†
c2_4_10	11/3825.67	12.20/-†	12/-†	12/-†	12/-†	rc1_4_10	36/7596.04	37/-†	36/+12.67%	36/+14.18%	36/+14.65%
c2_4_2	12/3929.89	12/+19.18%	12/+0.54%	12/+0.60%	12/+0.76%	rc1_4_2	36/7892.52	37/-†	36/+20.51%	36/+17.61%	36/+19.84%
c2_4_3	11/4018.02	12/-†	12/-†	12/-†	12/-†	rc1_4_3	36/7533.05	36.60/-†	36/+23.02%	36/+21.23%	36/+20.22%
c2_4_4	11/3702.49	12/-†	12/-†	12/-†	12/-†	rc1_4_4	36/7308.55	36.20/-†	36/+13.18%	36/+13.85%	36/+13.60%
c2_4_5	12/3938.69	13/-†	12/+0.63%	12/+0.68%	12.20/-†	rc1_4_5	36/8172.64	37.80/-†	36/+23.29%	36.20/-†	36.20/-†
c2_4_6	12/3875.94	13/-†	12.40/-†	12.40/-†	12.20/-†	rc1_4_6	36/8164.98	37.60/-†	36.20/-†	36.20/-†	36.40/-†
c2_4_7	12/3894.13	13/-†	12.60/-†	12.60/-†	12.60/-†	rc1_4_7	36/7948.51	37/-†	36/+19.41%	36/+21.63%	36/+20.67%
c2_4_8	11/4233.20	12.60/-†	12/-†	12/-†	12/-†	rc1_4_8	36/7772.57	37/-†	36/+18.86%	36/+20.80%	36/+15.91%
c2_4_9	12/3864.68	13/-†	12.80/-†	12.80/-†	13/-†	rc1_4_9	36/7733.35	37/-†	36/+16.74%	36/+18.75%	36/+18.70%
r1_4_1	40/10372.31	40/+18.50%	40/+4.91%	40/+4.82%	40/+4.88%	rc2_4_1	11/6682.37	12.80/-†	13/-†	13/-†	12.80/-†
r1_4_10	36/8094.10	36.60/-†	36/+14.45%	36/+14.10%	36/+15.74%	rc2_4_10	8/4278.61	8/+22.13%	8/+8.13%	8/+8.13%	8/+9.40%
r1_4_2	36/8898.15	36.40/-†	36/+25.76%	36/+26.78%	36/+21.74%	rc2_4_2	9/6180.62	12/-†	10.40/-†	10.40/-†	10.40/-†
r1_4_3	36/7808.34	36/+29.42%	36/+19.62%	36/+20.70%	36/+15.13%	rc2_4_3	8/4930.84	9/-†	9/-†	9/-†	8.80/-†
r1_4_4	36/7282.78	36/+16.81%	36/+12.57%	36/+12.63%	36/+10.94%	rc2_4_4	8/3631.01	8/+18.70%	8/+7.42%	8/+7.42%	8/+8.64%
r1_4_5	36/9222.96	37/-†	36/+18.85%	36/+24.66%	36/+20.87%	rc2_4_5	8/6706.44	11/-†	10/-†	10/-†	10/-†
r1_4_6	36/8358.69	36.40/-†	36/+19.60%	36/+18.54%	36/+19.16%	rc2_4_6	8/5766.61	10/-†	9/-†	9/-†	9/-†
r1_4_7	36/7616.15	36/+27.20%	36/+16.28%	36/+14.69%	36/+14.88%	rc2_4_7	8/5334.72	9/-†	9/-†	9/-†	8.80/-†
r1_4_8	36/7257.28	36/+15.94%	36/+11.27%	36/+11.07%	36/+9.47%	rc2_4_8	8/4790.14	8/+16.43%	8/+7.30%	8/+6.55%	8/+5.19%
r1_4_9	36/8694.78	36.80/-†	36/+12.16%	36/+16.51%	36/+24.16%	rc2_4_9	8/4551.11	8/+20.40%	8/+9.46%	8/+8.27%	8/+4.84%

ALNS-Base, $p=5.02 \times 10^{-14}$)—all under zero-shot transfer from synthetic training data.

Future work will extend the framework with graph neural network guidance for scalable heuristic acceleration (companion study), replace handcrafted state features with attention-based graph encodings [12], [13], and explore application to dynamic, real-time routing fleets.

ACKNOWLEDGMENT

The authors thank Dr. Ho Thi Linh for her guidance throughout this work.

REFERENCES

- [1] G. B. Dantzig and J. H. Ramser, “The truck dispatching problem,” *Management Science*, vol. 6, no. 1, pp. 80–91, 1959.
- [2] S. Ropke and D. Pisinger, “An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows,” *Transportation Science*, vol. 40, no. 4, pp. 455–472, 2006.
- [3] P. Shaw, “Using constraint programming and local search methods to solve vehicle routing problems,” in *Principles and Practice of Constraint Programming (CP 1998)*, LNCS 1520, pp. 417–431, Springer, 1998.
- [4] D. Pisinger and S. Ropke, “A general heuristic for vehicle routing problems,” *Computers & Operations Research*, vol. 34, no. 8, pp. 2403–2435, 2007.
- [5] M. M. Solomon, “Algorithms for the vehicle routing and scheduling problems with time window constraints,” *Operations Research*, vol. 35, no. 2, pp. 254–265, 1987.
- [6] H. Gehring and J. Homberger, “A parallel two-phase meta-heuristic for routing problems with time windows,” in *Proc. Asia-Pacific Decision Sciences Institute Conf.*, pp. 114–124, 1999.
- [7] B. P. Welford, “Note on a method for calculating corrected sums of squares and products,” *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.
- [8] Z. Wang *et al.*, “Dueling network architectures for deep reinforcement learning,” in *Proc. 33rd ICML*, vol. 48, pp. 1995–2003, 2016.
- [9] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, “Prioritized experience replay,” in *Int. Conf. Learning Representations (ICLR)*, 2016.
- [10] J. Tobin *et al.*, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *Proc. IEEE/RSJ IROS*, pp. 23–30, 2017.
- [11] W. R. Thompson, “On the likelihood that one unknown probability exceeds another in view of the evidence of two samples,” *Biometrika*, vol. 25, no. 3/4, pp. 285–294, 1933.
- [12] O. Vinyals, M. Fortunato, and N. Jaitly, “Pointer networks,” in *Advances in NeurIPS*, vol. 28, pp. 2692–2700, 2015.
- [13] W. Kool, H. van Hoof, and M. Welling, “Attention, learn to solve routing problems!” in *Int. Conf. Learning Representations (ICLR)*, 2019.
- [14] M. Nazari, A. Oroojlooy, L. V. Snyder, and M. Takáč, “Reinforcement learning for solving the vehicle routing problem,” in *Advances in NeurIPS*, pp. 9861–9871, 2018.
- [15] Google LLC, “OR-Tools: Open source software for combinatorial optimization,” 2023. [Online]. Available: <https://developers.google.com/optimization>
- [16] Y. Nagata and O. Bräysy, “A powerful route minimization heuristic for the vehicle routing problem with time windows,” *Operations Research Letters*, vol. 37, no. 5, pp. 333–338, 2009.
- [17] Y. Nagata, O. Bräysy, and W. Dullaert, “A penalty-based edge assembly memetic algorithm for the vehicle routing problem with time windows,” *Computers & Operations Research*, vol. 37, no. 4, pp. 724–737, 2010.